

# vuln\_lkm\_cli.pdf

---

*How the Vuln-LKM userspace CLI works*

Repository: [github.com/mayday5000/Vuln-LKM](https://github.com/mayday5000/Vuln-LKM)

Binary: vuln\_lkm\_cli (gcc -Wall -O2)

Device: /dev/vuln\_lkm

Shared ABI: vuln\_lkm.h

*Educational material for the Vuln-LKM loadable kernel module. Not an exploit guide. Integer wrap and a bounded-struct string copy are described as they appear in the source.*

## 1. Role of the CLI

vuln\_lkm\_cli is ordinary userspace. It is not a kernel module, not a udev helper, not a test framework. It opens /dev/vuln\_lkm, fills the structs defined in vuln\_lkm.h, and calls ioctl(2). That is the entire contract. The kernel module never sees argv, never sees the interactive menu, never sees the base64 dump. Those exist so a human can watch the ABI on the way out.

The program is built by the same Makefile that builds the .ko: gcc -Wall -O2 -o vuln\_lkm\_cli vuln\_lkm\_cli.c. It includes vuln\_lkm.h with \_\_KERNEL\_\_ undefined, so the userspace half of the header pulls sys/ioctl.h and typedefs \_\_u32 as uint32\_t. If you rebuild the header's ioctl numbers without rebuilding the CLI, the command words will not match the module and you will get ENOTTY.

There are two front ends in one binary. argc == 1 enters an interactive loop. argc >= 2 is a one-shot get|set|add|sub|str. -h and --help print help and exit without opening the device. Every successful or attempted ioctl prints the command word in hex, the integer or string payload, and a base64 encoding of the struct bytes.

## 2. The shared header as the ABI

The CLI does not invent ioctl numbers. It uses the macros from vuln\_lkm.h. Those macros expand at compile time to 32-bit command words. On Linux, \_IOR and \_IOW pack direction, type (magic 0x1B), ordinal, and sizeof the third argument.

```
#ifndef VULN_LKM_H
#define VULN_LKM_H

#ifdef __KERNEL__
#include <linux/ioctl.h>
#include <linux/types.h>
#else
#include <sys/ioctl.h>
#include <stdint.h>
typedef uint32_t __u32;
#endif

#define VULN_LKM_NAME "vuln_lkm"
#define VULN_LKM_IOC_MAGIC 0x1B
#define VULN_LKM_STR_MAX 256
#define VULN_LKM_KBUF 32

struct vuln_lkm_int {
    __u32 value;
};

struct vuln_lkm_str {
    __u32 len;
    char data[VULN_LKM_STR_MAX];
};

#define VULN_LKM_GET _IOR(VULN_LKM_IOC_MAGIC, 1, struct vuln_lkm_int)
#define VULN_LKM_SET _IOW(VULN_LKM_IOC_MAGIC, 2, struct vuln_lkm_int)
#define VULN_LKM_ADD _IOW(VULN_LKM_IOC_MAGIC, 3, struct vuln_lkm_int)
#define VULN_LKM_SUB _IOW(VULN_LKM_IOC_MAGIC, 4, struct vuln_lkm_int)
#define VULN_LKM_STR _IOW(VULN_LKM_IOC_MAGIC, 5, struct vuln_lkm_str)

#endif
```

Expected packing on a typical 64-bit Linux userspace (dir in bits 30–31, size in 16–29, type in 8–15, nr in 0–7):

- VULN\_LKM\_GET = \_IOR(0x1B, 1, struct vuln\_lkm\_int) → 0x80041b01 (read, 4 bytes)
- VULN\_LKM\_SET = \_IOW(0x1B, 2, struct vuln\_lkm\_int) → 0x40041b02 (write, 4 bytes)
- VULN\_LKM\_ADD = \_IOW(0x1B, 3, struct vuln\_lkm\_int) → 0x40041b03
- VULN\_LKM\_SUB = \_IOW(0x1B, 4, struct vuln\_lkm\_int) → 0x40041b04

- VULN\_LKM\_STR = \_IOW(0x1B, 5, struct vuln\_lkm\_str) → 0x41041b05 (write, 260 bytes)

The CLI prints ioctl cmd: 0x%x of whatever libc produced. If you ever see a different hex, your headers or architecture packed \_IOC differently. The kernel switch compares the full 32-bit word; a size mismatch is a different command.

### 3. Opening the device

open\_dev() is open("/dev/vuln\_lkm", O\_RDWR). The path is "/dev/" VULN\_LKM\_NAME so it cannot drift from the miscdevice name. O\_RDWR is conventional for ioctl; this driver does not implement .read or .write, so the mode mainly has to pass the inode permission check. load.sh chmods the node 0666 after insmod. If the module is not loaded, open fails with ENOENT. If permissions are 0600 and you are not root, open fails with EACCES. The CLI prints open /dev/vuln\_lkm: <strerror> and exits 1. It does not retry, does not look for another node, does not fall back to /sys.

The file descriptor is held for the lifetime of main. Interactive mode reuses one fd for every menu pick. One-shot mode opens, issues one ioctl family, closes. There is no flock. Two CLI processes can race on g\_val in the kernel; the lab is meant to be used one at a time.

### 4. Integer commands: GET, SET, ADD, SUB

All four share struct vuln\_lkm\_int { uint32\_t value; } and the helper do\_int(). do\_int prints the struct (show\_int), calls ioctl(fd, cmd, arg), and reports ioctl ok or ioctl failed: errno. The pointer passed to ioctl is the address of a local struct. The kernel will copy\_from\_user or copy\_to\_user that address. The CLI never inspects the kernel's g\_val except by GET, which copy\_to\_user's into the same struct.

#### 4.1 parse\_u32 and strtoul base 0

Numbers are parsed with strtoul(s, &end, 0). Base 0 means a 0x prefix is hex, a leading 0 is octal, otherwise decimal. That is why the help text says <n> is decimal or 0xhex and why set 0xffffffffe works. Overflow of unsigned long is reported via errno. A leftover character after the number is a parse error. The result is truncated to uint32\_t. Values larger than 2<sup>32</sup>-1 on a 64-bit host will silently truncate; keep inputs in the u32 range.

#### 4.2 GET

cmd\_get zeros a vuln\_lkm\_int, calls do\_int with sending=0 so the banner says number buffer rather than number sent, then if ioctl succeeded prints kernel value and a second base64 of the now-filled struct. Before ioctl the four bytes are 0. After ioctl they are whatever the kernel wrote. Compare the two b64 lines if you want to see the copy\_to\_user.

#### 4.3 SET, ADD, SUB

Each constructs arg.value = v and calls do\_int with sending=1. SET plants g\_val. ADD asks the kernel to add. SUB asks the kernel to subtract. The CLI does not clamp, does not warn about wrap, does not read back unless you GET afterwards. Wrap is a kernel-side printk and a later GET. Example: set 10 then sub 11 then get should show 4294967295. set 0xffffffffe then add 3 then get should show 1.

### 5. STR command

cmd\_str is the only path that uses struct vuln\_lkm\_str. It memset's the 260-byte object to zero, caps strlen(s) at VULN\_LKM\_STR\_MAX (256), stores that as len, memcpy's the bytes into data[], prints show\_str, then ioctl(VULN\_LKM\_STR, &arg). The CLI cap is 256, matching the ioctl blob, not 32. Sending a 40-character string is allowed by the CLI. The kernel memcpy's 40 bytes into a 32-byte kbuf. That is the lab.

show\_str prints the C string with %s. If data is not NUL-terminated inside the 256-byte array, %s would run off; memset of the whole struct plus a cap at 256 with no forced terminator at data[len] is almost safe because the remainder is zeros unless len == 256. A 256-byte payload with no room for a terminator can make %s read past data[]. The ioctl still sends 260 bytes of struct. Interactive mode uses fgets into a 256-byte buffer and strips a newline, so interactive strings are shorter than 256.

The 40-A example in --help is the intended overflow trigger: forty 0x41 bytes, len=40, kernel kbuf[32]. Run it in QEMU.

## 6. Base64 dump

print\_b64 is a local encoder. Alphabet is the standard RFC 4648 table. It walks the buffer in 3-byte groups, emits 4 characters, pads with =. For GET/SET/ADD/SUB the payload is 4 bytes, so the b64 is 8 characters with padding. For STR the payload is the entire 260-byte struct (len plus 256 data bytes), so the dump is long. That is deliberate: you can paste the b64 into a decoder and see the little-endian len word followed by the string bytes.

Nothing about base64 is sent to the kernel. It is a display of the bytes the ioctl pointer refers to. If you ever need to prove what userspace handed copy\_from\_user, this dump is that proof.

## 7. Interactive loop

interactive() prints a seven-line menu and reads one line with fgets into 32 bytes. Only line[0] is the command. 1 GET, 2 SET, 3 ADD, 4 SUB, 5 STR, h help, q quit. SET/ADD/SUB call read\_u32 which fgets a 64-byte line and parse\_u32. STR fgets into a 256-byte buffer (VULN\_LKM\_STR\_MAX), strips a trailing newline, and cmd\_str. Unknown keys print unknown option. EOF (Ctrl-D) leaves the loop.

The menu text says overflow wrap and copy into 32-byte kernel buffer so the operator is not surprised. Help from h reprints print\_help. The interactive path and the argv path share cmd\_get / cmd\_set / cmd\_add / cmd\_sub / cmd\_str. There is no second implementation.

## 8. argv dispatch in main

Order of checks: help without open; open\_dev or exit 1; argc==1 interactive; else exact matches: get with argc==2, set/add/sub with argc==3 and a successful parse, str with argc==3 (the string is argv[2], so a string with spaces must be quoted by the shell). Anything else prints help and returns 1. close(fd) always runs. A negative rc from ioctl helpers becomes exit status 1.

str with no argument, set with a non-number, extra arguments, and unknown verbs all fall through to help. There is no subcommand for dump-only or dry-run. There is no way to send a crafted len that differs from strlen; the CLI always sets len from the C string length. To send a mismatched len you would write a different program. This CLI is a tester, not a packet forger.

## 9. What the CLI does not do

- No privilege-escalation payload, no SMEP/KASLR bypass, no ROP.
- No raw ioctl number on the command line; only the five named ops.
- No read(2)/write(2) on the device (the driver has no .read/.write).
- No mmap, poll, or netlink.
- No retry on EINTR; a signal during ioctl surfaces as failure.
- No JSON, no scriptable machine output beyond the printed hex and b64.

## 10. Example sessions (userspace side)

These are the prints the CLI is written to produce. Kernel dmesg is a separate stream (see vuln\_lkm.pdf and the run-debug manual).

### 10.1 GET after a fresh insmod

```
$ ./vuln_lkm_cli get
```

```
--- GET ---
ioctl cmd: 0x80041b01
number buffer: 0 (0x00000000)
payload b64: AAAAAA==
ioctl ok
kernel value: 100 (0x00000064)
returned b64: ZAAAAA==
```

g\_val starts at 100. Little-endian 100 is 64 00 00 00, which is b64 ZAAAAA==. The pre-ioctl buffer was zeros (AAAAAA==).

### 10.2 Underflow wrap

```
$ ./vuln_lkm_cli set 10
--- SET ---
ioctl cmd: 0x40041b02
number sent: 10 (0x0000000a)
payload b64: CgAAAA==
ioctl ok

$ ./vuln_lkm_cli sub 11
--- SUB ---
ioctl cmd: 0x40041b04
number sent: 11 (0x0000000b)
payload b64: CwAAAA==
ioctl ok

$ ./vuln_lkm_cli get
kernel value: 4294967295 (0xffffffff)
```

### 10.3 Overflow wrap

```
$ ./vuln_lkm_cli set 0xfffffffffe
--- SET ---
ioctl cmd: 0x40041b02
number sent: 4294967294 (0xfffffffffe)
payload b64: /v///w==
ioctl ok

$ ./vuln_lkm_cli add 3
--- ADD ---
ioctl cmd: 0x40041b03
number sent: 3 (0x00000003)
payload b64: AwAAAA==
ioctl ok

$ ./vuln_lkm_cli get
kernel value: 1 (0x00000001)
```

## 10.4 STR that fits, then STR that does not

```
$ ./vuln_lkm_cli str hello
--- STR ---
ioctl cmd: 0x41041b05
string sent: "hello"
len sent: 5
payload b64: BQAAAGhlbGxv... # 260-byte struct, truncated here
ioctl ok

$ ./vuln_lkm_cli str AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
--- STR ---
ioctl cmd: 0x41041b05
string sent: "AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
len sent: 40
ioctl ok
# kernel memcpy(40) into kbuf[32]; guest may oops. Use QEMU.
```

## 11. Error paths the CLI surfaces

open failure: printed to stderr, exit 1. ioctl failure: strerror(errno) on stdout, helper returns -1, main maps that to exit 1. Typical errno values: ENOTTY if the command word does not match the loaded module (header skew), EFAULT if the pointer were bad (the CLI's locals are always valid; you would need a different caller), ENODEV/ENXIO if the misc device vanished mid-run after rmmmod. A guest oops during STR may kill the CLI with a signal or hang the VM; that is why STR length tests belong in QEMU.

## 12. Complete source listing: vuln\_lkm\_cli.c

The file as committed. Helpers: print\_help, print\_b64, show\_int, show\_str, do\_int, parse\_u32, read\_u32, open\_dev, cmd\_\*, interactive, main.

```
#include <errno.h>
#include <fcntl.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "vuln_lkm.h"

#define DEV_PATH "/dev/" VULN_LKM_NAME

static const char b64tab[] =
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/";

static void print_help(const char *argv0)
{
    printf(
        "vuln_lkm_cli - userspace tester for the vuln_lkm educational LKM\n"
        "\n"
        "Usage:\n"
        "  %s                               interactive menu\n"
        "  %s -h|--help                     this help\n"
        "  %s get                           VULN_LKM_GET\n"
        "  %s set <n>                       VULN_LKM_SET\n"
        "  %s add <n>                       VULN_LKM_ADD (u32 wrap)\n"
        "  %s sub <n>                       VULN_LKM_SUB (u32 wrap)\n"
        "  %s str <string>                 VULN_LKM_STR (copy into 32-byte kbuf)\n"
        "\n"
        "<n> is decimal or 0xhex.\n"
        "Every ioctl prints cmd hex, the number or string, and base64 of the payload.\n"
        "Device: %s\n"
        "\n"
        "Examples:\n"
```

```

" %s get\n"
" %s set 10\n"
" %s sub 11          # 10-11 wraps to 4294967295\n"
" %s set 0xffffffffe\n"
" %s add 3           # wraps to 1\n"
" %s str hello       # fits in 32-byte kbuf\n"
" %s str AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA # 40 bytes > 32\n"
"\n"
"Interactive keys: 1 GET  2 SET  3 ADD  4 SUB  5 STR  q quit\n",
■■argv0, argv0, argv0, argv0, argv0, argv0, argv0,
■■DEV_PATH,
■■argv0, argv0, argv0, argv0, argv0, argv0, argv0);
}

static void print_b64(const void *data, size_t n)
{
    ■const unsigned char *p = data;
    ■size_t i;

    ■for (i = 0; i < n; i += 3) {
        ■■unsigned a = p[i];
        ■■unsigned b = (i + 1 < n) ? p[i + 1] : 0;
        ■■unsigned c = (i + 2 < n) ? p[i + 2] : 0;
        ■■unsigned triple = (a << 16) | (b << 8) | c;

        ■■putchar(b64tab[(triple >> 18) & 63]);
        ■■putchar(b64tab[(triple >> 12) & 63]);
        ■■putchar((i + 1 < n) ? b64tab[(triple >> 6) & 63] : '=');
        ■■putchar((i + 2 < n) ? b64tab[triple & 63] : '=');
        ■}
    }

    static void show_int(const char *op, unsigned int cmd,
        ■■const struct vuln_lkm_int *arg, int sending)
    {
        ■printf("\n--- %s ---\n", op);
        ■printf("ioctl cmd: 0x%x\n", cmd);
        ■printf("number %s: %u (0x%08x)\n",
            ■sending ? "sent" : "buffer", arg->value, arg->value);
        ■printf("payload b64: ");
        ■print_b64(arg, sizeof(*arg));
        ■printf("\n");
    }

    static void show_str(unsigned int cmd, const struct vuln_lkm_str *arg)
    {
        ■printf("\n--- STR ---\n");
        ■printf("ioctl cmd: 0x%x\n", cmd);
        ■printf("string sent: \"%s\"\n", arg->data);
        ■printf("len sent: %u\n", arg->len);
        ■printf("payload b64: ");
        ■print_b64(arg, sizeof(*arg));
        ■printf("\n");
    }

    static int do_int(int fd, const char *op, unsigned int cmd,
        ■■struct vuln_lkm_int *arg, int sending)
    {
        ■show_int(op, cmd, arg, sending);
        ■if (ioctl(fd, cmd, arg) < 0) {
            ■■printf("ioctl failed: %s\n", strerror(errno));
            ■■return -1;
            ■}
        ■printf("ioctl ok\n");
        ■return 0;
    }

    static int parse_u32(const char *s, uint32_t *out)
    {
        ■unsigned long v;
        ■char *end;

```

```

■errno = 0;
■v = strtoul(s, &end, 0);
■if (end == s || *end != '\0' || errno)
■return -1;
■*out = (uint32_t)v;
■return 0;
}

static int read_u32(const char *prompt, uint32_t *out)
{
■char buf[64];

■printf("%s", prompt);
■fflush(stdout);
■if (!fgets(buf, sizeof(buf), stdin))
■return -1;
■buf[strcspn(buf, "\n")] = '\0';
■return parse_u32(buf, out);
}

static int open_dev(void)
{
■int fd = open(DEV_PATH, O_RDWR);

■if (fd < 0)
■fprintf(stderr, "open %s: %s\n", DEV_PATH, strerror(errno));
■return fd;
}

static int cmd_get(int fd)
{
■struct vuln_lkm_int arg = {0};

■if (do_int(fd, "GET", VULN_LKM_GET, &arg, 0) == 0) {
■printf("kernel value: %u (0x%08x)\n", arg.value, arg.value);
■printf("returned b64: ");
■print_b64(&arg, sizeof(arg));
■printf("\n");
■}
■return 0;
}

static int cmd_set(int fd, uint32_t v)
{
■struct vuln_lkm_int arg = { .value = v };

■return do_int(fd, "SET", VULN_LKM_SET, &arg, 1);
}

static int cmd_add(int fd, uint32_t v)
{
■struct vuln_lkm_int arg = { .value = v };

■return do_int(fd, "ADD", VULN_LKM_ADD, &arg, 1);
}

static int cmd_sub(int fd, uint32_t v)
{
■struct vuln_lkm_int arg = { .value = v };

■return do_int(fd, "SUB", VULN_LKM_SUB, &arg, 1);
}

static int cmd_str(int fd, const char *s)
{
■struct vuln_lkm_str arg;
■size_t n = strlen(s);

■memset(&arg, 0, sizeof(arg));
■if (n > VULN_LKM_STR_MAX)
■n = VULN_LKM_STR_MAX;

```

```

    arg.len = (uint32_t)n;
    memcpy(arg.data, s, n);
    show_str(VULN_LKM_STR, &arg);
    if (ioctl(fd, VULN_LKM_STR, &arg) < 0) {
        printf("ioctl failed: %s\n", strerror(errno));
        return -1;
    }
    printf("ioctl ok\n");
    return 0;
}

static int interactive(int fd)
{
    char line[32];
    uint32_t v;
    char buf[VULN_LKM_STR_MAX];

    while (1) {
        puts("\nvuln_lkm CLI");
        puts(" 1) GET integer");
        puts(" 2) SET integer");
        puts(" 3) ADD (u32 overflow wrap)");
        puts(" 4) SUB (u32 underflow wrap)");
        puts(" 5) STR (copy into 32-byte kernel buffer)");
        puts(" h) help");
        puts(" q) quit");
        printf("> ");
        fflush(stdout);
        if (!fgets(line, sizeof(line), stdin))
            break;
        switch (line[0]) {
            case '1':
                cmd_get(fd);
                break;
            case '2':
                if (read_u32("value to SET (dec or 0xhex): ", &v))
                    puts("bad number");
                else
                    cmd_set(fd, v);
                break;
            case '3':
                if (read_u32("value to ADD (dec or 0xhex): ", &v))
                    puts("bad number");
                else
                    cmd_add(fd, v);
                break;
            case '4':
                if (read_u32("value to SUB (dec or 0xhex): ", &v))
                    puts("bad number");
                else
                    cmd_sub(fd, v);
                break;
            case '5': {
                size_t n;

                printf("string to send: ");
                fflush(stdout);
                if (!fgets(buf, sizeof(buf), stdin))
                    break;
                n = strlen(buf);
                if (n && buf[n - 1] == '\n')
                    buf[--n] = '\0';
                cmd_str(fd, buf);
                break;
            }
            case 'h':
            case 'H':
                print_help("vuln_lkm_cli");
                break;
            case 'q':
            case 'Q':

```

```

    return 0;
default:
    puts("unknown option (h for help)");
    break;
}
}
return 0;
}

int main(int argc, char **argv)
{
    int fd, rc = 0;
    uint32_t v;

    if (argc >= 2 && (!strcmp(argv[1], "-h") || !strcmp(argv[1], "--help"))) {
        print_help(argv[0]);
        return 0;
    }

    fd = open_dev();
    if (fd < 0)
        return 1;

    if (argc == 1) {
        rc = interactive(fd);
    } else if (!strcmp(argv[1], "get") && argc == 2) {
        rc = cmd_get(fd);
    } else if (!strcmp(argv[1], "set") && argc == 3 && !parse_u32(argv[2], &v)) {
        rc = cmd_set(fd, v);
    } else if (!strcmp(argv[1], "add") && argc == 3 && !parse_u32(argv[2], &v)) {
        rc = cmd_add(fd, v);
    } else if (!strcmp(argv[1], "sub") && argc == 3 && !parse_u32(argv[2], &v)) {
        rc = cmd_sub(fd, v);
    } else if (!strcmp(argv[1], "str") && argc == 3) {
        rc = cmd_str(fd, argv[2]);
    } else {
        print_help(argv[0]);
        rc = 1;
    }

    close(fd);
    return rc < 0 ? 1 : rc;
}

```

## 13. How this maps to kernel functions

- `cmd_get` → `ioctl GET` → `vuln_lkm_ioctl GET` branch → `copy_to_user g_val`
- `cmd_set` → `ioctl SET` → `store g_val`
- `cmd_add` → `ioctl ADD` → `vuln_lkm_add` (u32 wrap)
- `cmd_sub` → `ioctl SUB` → `vuln_lkm_sub` (u32 wrap)
- `cmd_str` → `ioctl STR` → `vuln_lkm_str` (memcpy into 32-byte kbuf)

GDB on the guest kernel will stop in those four symbols if you follow `vuln_lkm_run_debug.pdf`. The CLI itself you can also gdb on the host, but the interesting state (`g_val`, `kbuf`) is on the kernel stack and in module `.bss`.